

Search APIs Are Decision Surfaces for Tool-Using Language Agents

Anonymous ACL submission

Abstract

Search APIs have become essential in language-model agents and market is increasingly seeing many Search API offerings. Search-augmented language-model agents are often built as if commercial search APIs were interchangeable retrieval backends and while considering a replacement the go to measurement has been answer accuracy. We test that assumption by freezing the answer model, prompt, tools, iteration budget, and page-fetch backend, then swapping only the search provider. On 100 hard web-search questions from SEALQA-HARD, Brave, Tavily, and Firecrawl produce nearly indistinguishable exact-match accuracy: 21, 21, and 23 out of 100. We show that this apparent equivalence hides large differences in the agent’s internal retrieval economy. We introduce a per-URL oracle that labels every URL visible to the agent—fetched or not—and analyze search APIs as *decision surfaces*: ranked snippets and URLs that determine whether the agent answers, searches again, or spends fetch tokens. The same agent sees gold-supporting evidence on 33, 24, and 19 queries across the three providers; its decision partition shifts from rich-snippet reading, to rank-one exploitation, to broad blind exploration. A surface-only contamination metric, the contradict-to-gold URL ratio, varies from 0.92 to 2.59 without using the model’s answer. Provider choice is therefore a retrieval-budget choice, not only a recall choice.

1 Introduction

Search-augmented language agents have a familiar shape: the model calls a web search API, reads a ranked result list, optionally opens pages, and returns an answer. Engineering practice often treats the search API as a replaceable component: if two providers return reasonable top- k results and the answer accuracy is within the ballpark then behavior is assumed to be mostly unchanged. This paper argues that the relevant object is not only the set of

returned URLs. It is the *pre-fetch surface* on which the agent must decide what to do next.

A tool-using agent does not see a search index. It sees titles, snippets, ranks, URLs, and provider-specific metadata. Before a page is opened, these fields are the only evidence for deciding whether to answer immediately, reformulate the query, or spend tokens on `fetch_page`. We call this interface a **decision surface**. Under this view, two providers can have similar final answer accuracy while inducing different fetch policies, different exposure to contradictory snippets, and different cost/latency profiles.

We test this view in a controlled setting. A single frozen GPT-5.4 agent answers 100 SEALQA-HARD questions with two tools: `search_web` and `fetch_page`. The agent, prompt, maximum iteration budget, and page-fetch backend are fixed. The only experimental condition is the search provider: Brave, Tavily, or Firecrawl. Every URL visible to the agent is then labeled by a Kimi-K2.6 judge as gold-supporting, contradicting, supporting the model answer, or garbage. The resulting 6,869 valid per-URL judgments let us audit not just *what* the agent answered, but *what it could have done* at each provider-specific surface.

The headline accuracy result is deliberately unexciting: 21, 21, and 23 exact matches out of 100. The mechanism is not. Brave exposes the largest visible-support ceiling (33 queries), Tavily concentrates 15/31 snippet-gold rows at rank one, and Firecrawl reaches the highest EM despite the sparsest visible-support ceiling by inducing broad exploration. The cross-provider overlap is also small: only 9 queries are answered correctly by all three providers, while 20 are answered correctly by exactly one provider and 62 by none.

Contributions. We make four contributions.

1. We formalize commercial search APIs as *decision surfaces* for tool-using agents rather than

- static ranked-list retrievers.
2. We introduce a per-URL oracle protocol that judges every element that was visible to the model. This includes URL, rank, title, snippet, metadata and page content (if fetched).
 3. We define surface-visible support, a four-way decision partition (SMART, MISSED, BLIND, NO-OP), and a contradiction-to-gold metric that is independent of the model answer.
 4. We report a controlled three-provider study showing that EM parity masks large differences in retrieval budget, evidence visibility, contamination, and instance-level complementarity.

2 Related Work

Retrieval-augmented generation and open-domain QA. Retrieval-augmented generation (RAG) combines parametric language models with external non-parametric memory (Lewis et al., 2020; Guu et al., 2020). Open-domain QA systems have long measured whether a retriever supplies answer-bearing passages to a reader or generator (Karpukhin et al., 2020; Izacard and Grave, 2021; Izacard et al., 2023). Heterogeneous IR benchmarks such as BEIR broaden retriever evaluation across tasks and domains (Thakur et al., 2021). Our study is complementary: the retriever is a production web-search API, the reader is an agent that may choose whether to fetch, and the measurement target is the *decision interface* between search and action.

Tool-using and search-using language agents. WebGPT fine-tunes a model to browse and cite web evidence (Nakano et al., 2021). Self-Ask, ReAct, IRCOT, Toolformer, and Self-RAG study models that decide when and how to call tools or retrieval functions (Press et al., 2022; Yao et al., 2023; Trivedi et al., 2022; Schick et al., 2023; Asai et al., 2024). FreshLLMs shows that search augmentation helps with dynamic factual knowledge and that evidence order matters (Vu et al., 2023). Recent work on over-searching argues that search-augmented systems need cost-sensitive metrics, not only answer accuracy (Xie et al., 2026). We share the policy perspective but isolate a different variable: the commercial provider’s snippet and URL surface under a fixed agent.

Browsing and hard-search benchmarks. GAIA, WebArena, and BrowseComp evaluate agents on tasks requiring web navigation, tool

use, or persistent browsing (Mialon et al., 2024; Zhou et al., 2024; Wei et al., 2025). SEALQA targets fact-seeking questions where search results are conflicting, noisy, or unhelpful (Pham et al., 2025). These benchmarks mainly evaluate agent capability. We instead use hard QA as a probe for provider-induced decision surfaces.

Evidence, attribution, and LLM judges. Generative search systems can produce fluent answers with incomplete or inaccurate citation support (Liu et al., 2023a; Yue et al., 2023). Long-context work shows that simply providing more text does not guarantee robust use of evidence (Liu et al., 2024). LLM-as-judge methods provide scalable evaluation but have known biases and require careful rubrics (Zheng et al., 2023; Liu et al., 2023b). Our judge labels individual retrieved documents with constrained JSON fields, rather than scoring final answers only.

Commercial search APIs. Brave, Tavily, Firecrawl, and Jina Reader expose different product interfaces and optional extraction features (Brave Software, 2026; Tavily, 2026; Firecrawl, 2026; Jina AI, 2026). In our main condition, provider-side page content is disabled and a shared fetcher opens pages on demand. This lets us attribute differences to the provider’s ranked URL/snippet surface rather than to distinct extraction pipelines.

3 Experimental Protocol

Figure 1 summarizes the protocol.

Dataset. We use a deterministic proportional stratified sample of 100 questions from the 254-row SEALQA-HARD subset. The strata are freshness $\times$ search_results $\times$ topic. The sample is designed to preserve the distribution of fast-changing, slow-changing, never-changing, conflicting, unhelpful, and topical categories. Appendix B gives the full distribution included in the repository’s sampling diagnostics.

Agent. The agent has two tools. search_web accepts a query string and returns at most ten ranked results. Each result has a document identifier, rank, title, URL, domain, snippet, and provider metadata. fetch_page accepts a document identifier from a prior search result and returns extracted markdown plus fetch metadata. The model may search again, fetch any previously visible document, answer with

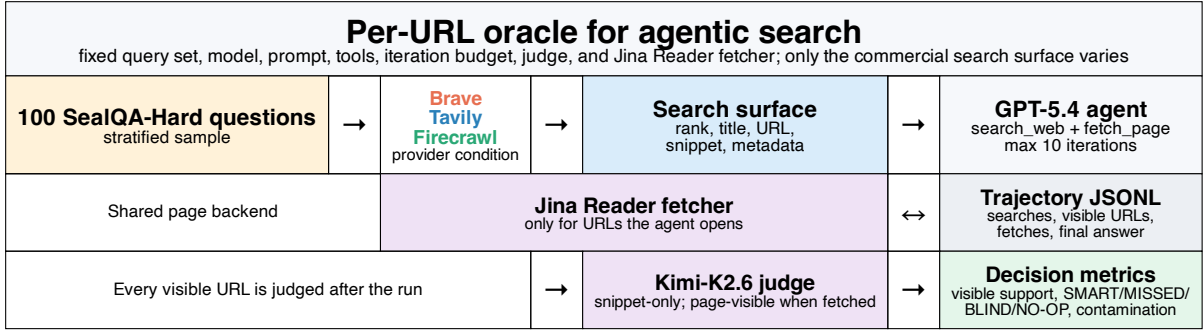


Figure 1: Experiment architecture and per-URL oracle. The same question set, model, prompt, tools, iteration budget, judge, and Jina Reader page-fetch backend are used in all conditions. Only the commercial search surface changes.

a short FINAL ANSWER, or abstain. The maximum iteration budget is 10.

Providers and normalization. We compare Brave, Tavily, and Firecrawl Search. Each adapter normalizes provider output into the same trace schema. Brave’s adapter preserves provider-native extra_snippets. Tavily is run with provider-side raw content disabled. Firecrawl is run without provider-side markdown scraping. All page text visible to the agent therefore comes from the same Jina Reader fetch backend. Our goal is to ensure we allow all providers to show comparable content to the model without one provider unfairly dumping large number of page content tokens into the context as in reality that remains infeasible at scale for RAG systems.

Traces. The experiment produces 300 provider-query trajectories. A trace records every search call, every returned URL, every fetch decision, fetch status, token usage, latency, final answer, and gold answer. The provider-comparison scripts deterministically derive EM/F1, token, URL-hit, and per-query support fields from these traces.

4 Per-URL Oracle and Metrics

Judge input and output. For every URL the agent saw, we run Kimi-K2.6 from Azure as the judge. The judge receives the question, gold answer, model final answer, and one retrieved document in the same XML-like format the answer model saw. For snippet-only records, the judge sees title, URL, domain, snippet, and metadata such as Brave extra_snippets. For page-visible records, it also sees extracted markdown. The required fields include contains_gold_answer, gold_answer_in_snippets,

gold_answer_in_extracted_page, contradicts_gold_answer, supports_model_answer, is_garbage, spans, and confidence. Across providers, 6,869 of 6,909 judge rows are valid; 40 invalid rows are excluded. Valid rows split into 6,519 snippet-only and 350 page-visible rows.

Surface-visible support. For a provider-query pair, define visible support as the event that at least one judged URL has contains_gold_answer true on either the snippet-only surface or a page-visible surface. This is a *lower bound* on provider pool support because unfetched pages are not page-judged.

Four-way decision partition. We join oracle labels with trace actions and place each provider-query pair into exactly one cell:

- SMART: visible support exists and the agent fetched a gold-supporting URL.
- MISSED: visible support exists and the agent fetched none of the gold-supporting URLs.
- BLIND: no visible support exists and the agent fetched at least one URL.
- NO-OP: no visible support exists and the agent fetched no URL.

The partition is descriptive, not causal. For example, a MISSED query may still be answered correctly from snippets.

Answer-independent contamination. A common misleading-evidence metric would count snippets that support the model’s wrong answer. That is partially circular because it depends on the model’s output. We therefore use a surface-only ratio:

$$r_{c:g} = \frac{|\{u : \text{contradicts_gold_answer}(u)\}|}{|\{u : \text{contains_gold_answer}(u)\}|},$$

computed over snippet-only URL rows. It measures contradicting evidence relative to supporting evidence on the surface, independent of the agent’s final answer.

5 Results

5.1 EM parity masks different evidence economies

Table 1 shows that final answer accuracy is nearly unchanged by provider choice. The EM spread is only two points across providers. The traditional recall-like signals are also close: exact gold URL hits are 59, 57, and 60; gold-domain hits are exactly 82 for each provider in this run. A static top- k view would therefore suggest that the providers are broadly interchangeable.

The evidence-economy view differs. Brave exposes more answer text in snippets and extra snippets (55 snippet hits and 71 extra-snippet hits), while Tavily and Firecrawl expose no extra-snippet field under our configuration. Firecrawl opens pages on more queries and has the largest page-answer-text count (61). Provider choice changes where answer evidence appears before the model decides whether to fetch.

5.2 The visible-support ceiling moves sharply

The per-URL judge reveals a larger difference than headline EM. Brave has visible support on 33 queries, compared with 24 for Tavily and 19 for Firecrawl. This is not just a count of URLs: it is the number of provider-query pairs for which the agent had at least one judged path to gold in the visible surface or in a page it actually opened.

This visible-support measure is intentionally conservative. It does not claim Brave’s true web pool contains more answer pages than Tavily’s or Firecrawl’s. It says that, under the fixed agent’s actual searches and fetches, the judged surface exposed gold support more often for Brave. Because only fetched pages are page-judged, hidden support in unfetched pages is undercounted for all providers.

5.3 The same agent enters different decision regimes

Figure 3 and Table 3 show the four-way decision partition. Tavily has the highest SMART EM rate: 7/11 (64%). This aligns with its rank-one concentration: 15/31 gold-supporting snippet rows are at rank one, making a top-result fetch more likely

to be useful. Brave has a much smaller SMART-MISSED gap: 38% versus 36%. This is consistent with a rich snippet surface where some answers can be read without fetching the supporting page. Firecrawl’s largest cell is BLIND: 67 queries, 16 of which are correct.

5.4 Contamination varies without looking at the answer

The contradict-to-gold ratio varies by $2.8\times$: 0.92 for Brave, 1.87 for Tavily, and 2.59 for Firecrawl. This ratio uses only snippet-only judge labels. It does not inspect page text and does not depend on what the model answered. It is therefore a useful provider-surface diagnostic: it estimates how much contradictory evidence the agent must navigate per unit of supporting evidence on the pre-fetch surface.

5.5 Aggregate EM hides instance-level complementarity

Figure 4 shows a second reason not to stop at EM. Although the providers have similar aggregate scores, only 9 questions are correct under all three. 9 are correct under exactly two providers; 20 are correct under exactly one provider; 62 are wrong under all three. Thus, provider choice changes which questions are answered, not only how many.

The pairwise rows in Appendix F show the same pattern: each pair has a non-trivial provider-specific slice. This suggests a practical opportunity for provider routing or surface-aware ensembling, but our current data are observational and too small to claim a deployed routing policy.

5.6 Many failures occur despite visible answer text

The provider-comparison traces also record whether the gold answer string appears anywhere in the retrieved text. This deterministic proxy is not the same as judge support, but it is useful for failure analysis. Answer text is visible somewhere in the retrieved text for 78, 75, and 76 queries. Yet the agent still gets only 21, 21, and 23 exact matches. The corresponding wrong-with-answer-text counts are 57, 56, and 53.

This reinforces the decision-surface argument. Provider surfaces do not merely decide whether evidence exists; they shape whether the model recognizes, trusts, and spends budget on the right evidence amid distractors.

Same final accuracy, different decision surfaces

Counts are queries unless URL rows are shown; bucket cells show n / exact-match count.

Brave	Tavily	Firecrawl
EM 21/100 F1 0.270	EM 21/100 F1 0.261	EM 23/100 F1 0.282
visible support 33 queries	visible support 24 queries	visible support 19 queries
rank-1 gold 12% (12/97)	rank-1 gold 48% (15/31)	rank-1 gold 11% (3/27)
contradict:gold 0.92 (89/97)	contradict:gold 1.87 (58/31)	contradict:gold 2.59 (70/27)
SMART 8/3 EM MISSED 25/9	SMART 11/7 EM MISSED 13/5	SMART 7/2 EM MISSED 12/5
BLIND 51/9 EM NO-OP 16/0	BLIND 55/9 EM NO-OP 21/0	BLIND 67/16 EM NO-OP 14/0

Figure 2: Provider profiles. The providers have similar final EM but different visible support, rank concentration, contamination, and decision partitions. Bucket entries report queries / exact-match count.

Provider	EM	F1	avg search	avg fetch	fetchd q.	tokens/query
Brave	21	0.270	2.29	1.02	65%	59,627
Tavily	21	0.261	2.74	1.30	76%	54,156
Firecrawl	23	0.282	2.51	1.28	81%	57,979
<i>Trace-derived support proxies</i>						
Brave	URL 59	domain 82	family 61	snippet 55	extra 71	page 52
Tavily	URL 57	domain 82	family 60	snippet 60	extra 0	page 57
Firecrawl	URL 60	domain 82	family 64	snippet 54	extra 0	page 61

Table 1: Headline metrics and deterministic trace-derived support proxies over 100 queries per provider. “Fetchd q.” is the fraction of queries on which the agent opened at least one page; avg fetch is valid fetch records per query. URL/domain/family rows count queries with a gold URL, gold domain, or source-family hit in retrieved results; snippet/extra/page rows count queries where the gold answer string appears in that surface according to deterministic trace analysis.

Provider	visible support	snippet rows	page rows	rank-1 gold	Brave	Tavily	Firecrawl
Brave	33	2,095	101	12/97 (12%)	8 / 3 (38%)	11 / 7 (64%)	7 / 2 (29%)
Tavily	24	2,339	125	15/31 (48%)	25 / 9 (36%)	13 / 5 (38%)	12 / 5 (42%)
Firecrawl	19	2,085	124	3/27 (11%)	51 / 9 (18%)	55 / 9 (16%)	67 / 16 (24%)
				NO-OP	16 / 0 (0%)	21 / 0 (0%)	14 / 0 (0%)
				SMART–MISSED gap	+2 pp	+26 pp	-13 pp

Table 2: Visible support and rank concentration from the per-URL judge. Rank-1 gold is computed over snippet-only gold-supporting URL rows, not over queries.

Table 3: Decision cells shown as n / EM count (EM rate). The SMART–MISSED gap is descriptive: it compares outcomes when support was visible and a gold-supporting URL was or was not fetched. It is not a randomized treatment effect.

6 Discussion

Provider choice is a policy choice. A search API changes the utility of the same fetch policy. Tavily’s rank-one concentration makes top-result fetching relatively valuable. Brave’s richer snippets reduce the marginal value of fetching some support-bearing pages. Firecrawl induces more broad exploration. A provider should therefore be paired with a provider-aware policy, not a fixed universal fetch heuristic.

Top-*k* relevance is incomplete for agents. Static relevance and gold URL hit rates remain useful, but they cannot see whether the agent had enough snippet evidence to answer, whether it should have fetched, or whether contradictory snippets competed with support. Decision-surface metrics complement IR metrics by measuring the evidence state available *before* a fetch action.

Decision partition: what the agent did with visible support				
Provider	visible support + fetched support	visible support + did not fetch support	no visible support + fetched	no visible support + no fetch
Brave	SMART 8 queries 3 EM (38%)	MISSED 25 queries 9 EM (36%)	BLIND 51 queries 9 EM (18%)	NO-OP 16 queries 0 EM (0%)
Tavily	SMART 11 queries 7 EM (64%)	MISSED 13 queries 5 EM (38%)	BLIND 55 queries 9 EM (16%)	NO-OP 21 queries 0 EM (0%)
Firecrawl	SMART 7 queries 2 EM (29%)	MISSED 12 queries 5 EM (42%)	BLIND 67 queries 16 EM (24%)	NO-OP 14 queries 0 EM (0%)

Figure 3: Four-way partition of each provider’s 100 queries. The rows sum to 100 within provider. The partition shows how provider surfaces induce different action regimes under the same agent.

Provider	gold rows	contradict rows	$r_{c:g}$
Brave	97	89	0.92
Tavily	31	58	1.87
Firecrawl	27	70	2.59

Table 4: Answer-independent surface contamination on snippet-only URL rows.

Aggregate view	Instance-level view	Implication
Brave 21/100	all 3 correct 9	Provider choice changes which questions are solvable. EM parity is not behavioral equivalence.
Tavily 21/100	exactly 2 correct 9	
Firecrawl 23/100	exactly 1 correct 20	
looks interchangeable	all wrong 62	

Figure 4: EM parity is not instance-level equivalence. Only 9 questions are solved by all providers, while 20 are solved by exactly one provider.

What providers could optimize. An agent-ready search API should optimize not only relevance but also actionability: calibrated snippets, low contradict-to-gold contamination, stable document identifiers, clear metadata about freshness and source type, and ranking that aligns answer-bearing pages with common agent fetch policies. These are testable surface properties.

What practitioners can use now. The metrics here can be computed offline from traces without rerunning provider APIs. A practitioner can run a small sample through candidate providers, judge visible URLs, and choose policies based on whether the provider behaves like a snippet-rich surface, a rank-concentrated surface, or a volume/exploration surface.

7 Threats to Validity

Single agent and judge. The decision partition is induced by one GPT-5.4 agent and one Kimi-K2.6 judge. A more aggressive fetch policy, another model family, or another judge could change

cell counts. The paper’s claim is therefore about provider-surface effects under a controlled agent, not a universal provider ranking.

Lower-bound support. Surface-visible support is a lower bound. We page-judge only fetched URLs, so an unfetched URL may contain gold on the page even when its snippet does not. This limitation is central to the interpretation and is why we avoid calling surface-visible support full provider recall.

Snippet asymmetry. Brave returns provider-native extra_snippets; Tavily and Firecrawl do not under our configuration. This is not a bug—the agent really sees those fields—but it means snippet-richness is partly a provider product feature. The contradiction-to-gold ratio is less sensitive to raw text volume because both numerator and denominator are computed on the same per-provider surface.

Observational policy analysis. The agent’s actions are observed, not randomized. We do not replay the same hidden state with result lists swapped, nor do we intervene on ranks, snippets, or fetch decisions. The current study establishes descriptive decision-surface diagnostics; causal replay is future work.

Scale and provider coverage. The main experiment has 100 questions and three providers. EM confidence intervals are wide, and small decision cells should be read as diagnostics rather than definitive effect sizes. Additional providers such as Bing, Google CSE, Serper, Exa, and others would broaden the claim.

8 Conclusion

Commercial search APIs are not interchangeable input pipes for tool-using language agents. In

our controlled study, final answer accuracy barely moves, but the internal retrieval economy changes sharply: visible support, rank concentration, contradiction, fetch behavior, and instance-level correctness all depend on the provider surface. Search APIs for agents should therefore be evaluated as decision surfaces. The operational question is not only which provider retrieves relevant URLs, but which provider induces the right retrieval-budget policy for a given agent.

Limitations

The main limitations are the single-agent/single-judge design, lower-bound page support, provider-specific snippet asymmetry, observational rather than causal policy analysis, and 100-query scale. These limitations constrain effect-size claims, but they do not remove the central observation that similar EM can arise from different decision policies.

Ethics Statement

This work evaluates commercial search APIs and LLM agents on public web-search QA data. We do not claim that one provider is universally better. Provider names are reported because they are experimental conditions and because practitioners need to reproduce the surface differences. The study may help reduce unnecessary page fetches and improve handling of contradictory evidence. It could also be overfit to a benchmark or used as a provider leaderboard without respecting the stated limitations; we discourage that use.

Reproducibility Statement

All paper numbers are deterministic functions of the committed query sample, trace JSONL files, judge JSONL files, and provider-comparison summaries. The raw trace and judge JSONLs are stored through Git LFS; run `git lfs pull` before executing `paper/figures/make_figures.py -audit`. Render-only mode regenerates figures from last validated constants and makes no model or provider API calls.

References

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations*.

Brave Software. 2026. Brave search api reference. <https://api-dashboard.search.brave.com/api-reference/>.

Firecrawl. 2026. Firecrawl search api documentation. <https://docs.firecrawl.dev/api-reference/endpoint/search>.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. REALM: Retrieval-augmented language model pre-training. In *Proceedings of the 37th International Conference on Machine Learning*.

Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2023. ATLAS: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research*, 24:1–43.

Gautier Izacard and Edouard Grave. 2021. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, pages 874–880.

Jina AI. 2026. Jina reader: Url to llm-friendly input. <https://r.jina.ai/>.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Nelson F. Liu, Tianyi Zhang, and Percy Liang. 2023a. Evaluating verifiability in generative search engines. *arXiv preprint arXiv:2304.09848*.

Yang Liu, Dan Iter, Yichong Xu, Shuhang Wang, Ruochen Xu, and Chenguang Zhu. 2023b. G-Eval: NLG evaluation using GPT-4 with better human alignment. *arXiv preprint arXiv:2303.16634*.

Gregoire Mialon, Clementine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. GAIA: A benchmark for general ai assistants. In *International Conference on Learning Representations*.

517	Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Benjamin Chess, and John Schulman. 2021. WebGPT: Browser-assisted question-answering with human feedback. <i>arXiv preprint arXiv:2112.09332</i> .	574
518		575
519		576
520		577
521		
522		578
523		579
524		580
525		581
526	Thinh Pham, Nguyen Nguyen, Pratibha Zunjare, Weiyuan Chen, Yu-Min Tseng, and Tu Vu. 2025. SealQA: Raising the bar for reasoning in search-augmented language models. <i>arXiv preprint arXiv:2506.01062</i> .	582
527		583
528		
529		584
530		585
531	Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. 2022. Measuring and narrowing the compositionality gap in language models. <i>arXiv preprint arXiv:2210.03350</i> .	586
532		587
533		588
534		589
535		
536	Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In <i>Advances in Neural Information Processing Systems</i> .	590
537		
538		591
539		592
540	Tavily. 2026. Tavily search api documentation. https://docs.tavily.com/ .	593
541		594
542		595
543	Nandan Thakur, Nils Reimers, Andreas Rucklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In <i>Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks</i> .	596
544		597
545		
546		598
547		599
548		600
549	Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. <i>arXiv preprint arXiv:2212.10509</i> .	601
550		602
551		603
552		604
553		605
554	Tu Vu, Mohit Iyyer, Xuezhi Wang, Noah Constant, Jerry Wei, Jason Wei, Chris Tar, Yun-Hsuan Sung, Denny Zhou, Quoc Le, and Thang Luong. 2023. FreshLLMs: Refreshing large language models with search engine augmentation. <i>arXiv preprint arXiv:2310.03214</i> .	606
555		
556		607
557		
558		608
559	Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. 2025. BrowseComp: A simple yet challenging benchmark for browsing agents. <i>arXiv preprint arXiv:2504.12516</i> .	609
560		610
561		611
562		612
563		613
564		614
565	Roy Xie, Deepak Gopinath, David Qiu, Dong Lin, Haitian Sun, Saloni Potdar, and Bhuwan Dhingra. 2026. Over-searching in search-augmented large language models. <i>arXiv preprint arXiv:2601.05503</i> .	615
566		616
567		617
568		618
569		619
570	Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In <i>International Conference on Learning Representations</i> .	620
571		
572		
573		
	Xiang Yue, Boshi Wang, Ziru Chen, Kai Zhang, Yu Su, and Huan Sun. 2023. Automatic evaluation of attribution by large language models. <i>arXiv preprint arXiv:2305.06311</i> .	
	Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Xing, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. <i>arXiv preprint arXiv:2306.05685</i> .	
	Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A realistic web environment for building autonomous agents. In <i>International Conference on Learning Representations</i> .	
	A Formal Metric Definitions	
	For provider p and query q , let $U_{p,q}$ be the set of visible URLs with valid judge rows. Let $G(u)$ be contains_gold_answer on either the snippet-only row or a page-visible row, $C(u)$ be contradicts_gold_answer on the snippet-only row, and $F(u)$ be whether the agent fetched URL u .	
	VisibleSupport(p, q) = $\mathbb{I}[\exists u \in U_{p,q} : G(u)]$	
	FetchGold(p, q) = $\mathbb{I}[\exists u \in U_{p,q} : F(u) \wedge G(u) \wedge u \text{ has page}]$	
	FetchAny(p, q) = $\mathbb{I}[\exists u \in U_{p,q} : F(u)]$.	
	The partition is then: SMART if VisibleSupport and FetchGold; MISSED if VisibleSupport and not FetchGold; BLIND if not VisibleSupport and FetchAny; NO-OP otherwise. The contradiction ratio is computed over snippet-only URL rows, not queries:	
	$r_{c:g}(p) = \frac{\sum_{u \in U_p^{\text{snippet}}} \mathbb{I}[C(u)]}{\sum_{u \in U_p^{\text{snippet}}} \mathbb{I}[G(u)]}.$	
	B Dataset Stratification	
	The repository sample file records dataset vtllms/sealqa, subset seal_hard, sample size 100, source size 254, seed 20260509, and strata fields freshness, search_results, and topic. The previous diagnostics report the following marginal distribution: 25 fast-changing, 43 slow-changing, and 32 never-changing questions; 57 conflicting and 43 unhelpful search-result labels; topics are Science & Technology 27, Sports 22, Entertainment 22, Others 12, Politics 9, and History & Geography 8. Effective years are 61 before 2024, 16 in 2024, and 23 in 2025. Question-type	

tags are multi-label: 72 advanced reasoning, 61 entity/event disambiguation, 13 temporal tracking, 5 cross-lingual, and 2 false-premise.

C Provider Request Configurations

Brave. The adapter sends GET `https://api.search.brave.com/res/v1/web/search` with `count=10`, `offset=0`, `country=US`, English language settings, `safesearch=moderate`, `spellcheck=true`, `text_decorations=false`, and `result_filter=web`. It maps `web.results[]` into rank, title, URL, snippet, domain, and provider metadata including `extra_snippets`.

Tavily. The adapter sends POST `https://api.tavily.com/search` with `search_depth=basic`, `topic=general`, `max_results=10`, and provider-side `answer/image/raw-content` options disabled.

Firecrawl. The adapter sends POST `https://api.firecrawl.dev/v2/search` with `limit=10`, `sources=[web]`, `country=US`, `timeout=60000`, and no provider-side markdown scraping in the main condition.

Page fetch. All provider conditions use Jina Reader through `fetch_page` for page text. Thus differences in page content arise through URL choice, not through distinct provider extractors.

D Judge Schema and Invalid Rows

The judge prompt requires JSON only:

```
{
  "contains_gold_answer": boolean,
  "gold_answer_in_snippets": boolean,
  "gold_answer_in_extracted_page": boolean,
  "supports_model_answer": boolean,
  "contradicts_gold_answer": boolean,
  "is_garbage": boolean,
  "garbage_reason": string,
  "answer_span": string,
  "gold_snippet_span": string,
  "gold_extracted_page_span": string,
  "confidence": number
}
```

A row is valid when it has the expected schema version, a parsed judgment dictionary, provider/query/retrieval/URL identifiers, and no execution or parse error. The audit script excludes invalid rows rather than treating them as negative evidence.

E Wilson Intervals for Decision Cells

F Pairwise Provider Complementarity

G Additional Token and Fetch Diagnostics

H Audit Manifest

The package includes `figures/decision_surface_audit.json` and `figures/decision_surface_audit.md`. Audit mode consumes the following files:

- `data/traces/phase1_v1_brave_gpt54_fetch_tool_jina_100.jsonl`
- `data/traces/phase1_v1_tavily_gpt54_fetch_tool_jina_100.jsonl`
- `data/traces/phase1_v1_firecrawl_gpt54_fetch_tool_jina_100.jsonl`
- `results/llm_judge/kimi_document_judge_surface_v3_brave_100_all_visible.jsonl`
- `results/llm_judge/kimi_document_judge_surface_v3_tavily_100_all_visible.jsonl`
- `results/llm_judge/kimi_document_judge_surface_v3_firecrawl_100_all_visible.jsonl`
- `results/provider_comparison/brave_tavily_firecrawl_fetch_tool_jina/provider_per_query.jsonl`
- `results/provider_comparison/brave_tavily_firecrawl_fetch_tool_jina/provider_summary.json`

The script checks for Git LFS pointer files and exits with an explicit error if raw data has not been pulled.

Cell	Brave	Tavily	Firecrawl
SMART	3/8: 38% [14–69]	7/11: 64% [35–85]	2/7: 29% [8–64]
MISSED	9/25: 36% [20–55]	5/13: 38% [18–64]	5/12: 42% [19–68]
BLIND	9/51: 18% [10–30]	9/55: 16% [9–28]	16/67: 24% [15–35]
NO-OP	0/16: 0% [0–19]	0/21: 0% [0–15]	0/14: 0% [0–22]

Table 5: Wilson 95% intervals for cell-level EM rates. The intervals are wide because some cells are small.

Pair	both correct	left only	right only	both wrong
Brave vs. Firecrawl	12	9	11	57
Brave vs. Tavily	12	9	9	57
Firecrawl vs. Tavily	12	11	9	61

Table 6: Pairwise exact-match overlap from the provider-comparison summary. Rows do not include the auxiliary “both wrong but one has answer text” diagnostic column, which is reported in the JSON audit file.

Provider	total tokens	median/query	max/query	>100k q.	fetch success/fail
Brave	5.96M	40,638	303,462	19	92/10
Tavily	5.42M	36,867	305,474	16	119/11
Firecrawl	5.80M	34,383	380,646	16	121/7

Table 7: Token and fetch diagnostics from provider summaries.